Time Complexity Analysis - USEI15 (Risk-Aware Shortest Paths)

1. Introduction

This document presents a detailed analysis of the time complexity of the algorithms implemented in **USEI15: Risk-Aware Shortest Paths.** The objective is to understand the system's performance as a function of the number of stations (V) and connections (E) in the railway network, in a context where edge costs can be negative (bonuses) and negative cycle detection is required.

2. Theoretical Foundations

The analysis is based on the following graph properties:

- ***V* (Vertices):** Total number of railway stations.
- ***E* (Edges):** Total number of connections (lines) between stations.

The system uses a **Map-based Adjacency List** to represent the graph, allowing for efficient neighbor lookups.

3. Algorithm Selection

Unlike USEI13, which uses **Dijkstra's algorithm** (suitable only for non-negative weights), USEI15 requires handling **negative costs** (bonuses for preferred links). Dijkstra's algorithm fails in these scenarios as it assumes that adding an edge never reduces the total path cost. Therefore, the **Bellman-Ford algorithm** was implemented. This algorithm can handle negative weights and detect the existence of negative cycles accessible from the source.

4. Complexity Analysis (Bellman-Ford)

4.1. Algorithm Phases

1. **Initialization (O(V)):** The algorithm begins by setting the distance to all stations as infinity (∞) and the distance to the source as 0. This requires iterating over all vertices.

2. **Edge Relaxation ($V \times E$):**

   - The core of the algorithm consists of an outer loop that executes at most **(V- 1)** times.
   - Within each iteration, the algorithm traverses all edges (***E***) of the graph to attempt to relax (improve) the distance to the destination.
   - **Implemented Optimization:** A "changed flag" was introduced. If no distances are updated during a complete iteration of the outer loop, the algorithm terminates immediately. This allows the complexity to drop to O(E) in the **best case** (if edges are favorably ordered).

3. **Negative Cycle Detection (O(E)):** After the relaxation phase, the algorithm performs a final pass over all edges (**E**). If any edge can still be relaxed in this phase, the existence of a negative cycle is confirmed.

4. **Path Reconstruction (O(V)):** If there are no negative cycles, the path is reconstructed by traversing the predecessors from the destination back to the source. In the worst case **(a linear graph)**, this traversal visits **V** vertices.

4.2 Total Complexity

The time complexity in the worst case is dominated by the relaxation phase:

O(V) + O(V x E) + O(E) + O(V) ≈ O(V x E)

Although it is asymptotically slower than Dijkstra **(O(E + V logV)** or **O(V^2)** depending on the implementation), this cost is necessary to ensure correctness (robustness) in graphs with negative weights.

5. Algorithm Determinism

As in US13, the processes involved are strictly deterministic.

- **Deterministic Aspects:** For the same graph state (nodes and edges with fixed costs) and the same source/destination station, the Bellman-Ford algorithm will always produce the same total cost value and the same sequence of stations.

- In the case of multiple paths with the exact same cost, the order of edge relaxation (based on the underlying data structure) will consistently determine which path is chosen.

6. Summary table

Operation / Phase	**Algorithm**	**Time Complexity (Worst Case)**	**Complexity (Best Case - Optimized)**
Initialization	Vertex Iteration	O(V)	O(V)
Path Calculation	Relaxation (Bellman-Ford)	O(V×E)	O(E)
Safety Check	Negative Cycle Detection	O(E)	O(E)
Result Construction	Predecessor Backtracking	O(V)	O(1)
Combined Total	Full Bellman-Ford	O(V×E)	O(E)